

IAA012 – FRA – Frameworks de IA

Aula 22 - 4.3 Resolução de exercício de Sistemas de Recomendação

Metadados

Prática 1 - Sistemas de Recomendação

1. Importação das Bibliotecas

O código utiliza:

- **tensorflow**: Para criar e treinar o modelo de recomendação.
- **keras.layers**: Inclui camadas de **Embedding**, **Dense**, e outras necessárias para o modelo de recomendação.
- **sklearn.utils.shuffle**: Para embaralhar os dados antes do treinamento.
- **numpy** e **pandas**: Para manipulação de arrays e dados tabulares.
- **matplotlib**: Para visualização gráfica.

2. Importação dos Dados

- **!wget** e **!unzip**: Comandos que baixam e descompactam o dataset de filmes.
- **pd.read_csv()**: Lê o arquivo CSV que contém as avaliações dos usuários para os filmes.

3. Conversão de **userId** e **movieId** para Categóricos

- **pd.Categorical()**: Converte as colunas **userId** e **movieId** para valores categóricos que podem ser usados em embeddings.
- O código também cria novas colunas **new_user_id** e **new_movie_id** para representar os IDs categorizados de usuários e filmes.

4. Criação do Modelo de Recomendação

- **Input()**: Define as entradas para os IDs de usuários e filmes.

- **Embedding()**: Camada de embedding usada para mapear IDs de usuários e filmes em vetores de dimensão **K=10**.
- **Concatenate()**: Junta os embeddings dos usuários e filmes em um único vetor.
- **Dense()**: Camada densa com 1024 unidades e ativação ReLU, seguida de uma camada de saída densa para prever a nota.

5. Compilação do Modelo

- **model.compile()**: Compila o modelo com a função de perda de erro quadrático médio (MSE) e o otimizador SGD com taxa de aprendizado de 0.08.

6. Separação dos Dados e Pré-processamento

- **shuffle()**: Embaralha os dados de usuários, filmes e avaliações.
- O dataset é dividido em 80% para treinamento e 20% para teste.
- As avaliações são centralizadas subtraindo a média das avaliações de treinamento.

7. Treinamento do Modelo

- **model.fit()**: Treina o modelo em 25 épocas, usando um tamanho de lote de 1024, com dados de validação.

8. Visualização da Função de Perda

- **plt.plot()**: Plota o gráfico da função de perda (**loss**) e perda de validação (**val_loss**) ao longo das épocas.

9. Recomendações para o Usuário

- O código gera recomendações para um usuário específico (neste caso, o usuário de ID 73023) criando um vetor com esse ID e repetindo-o para todos os filmes.
- **model.predict()**: Faz predições de notas para todos os filmes e identifica aquele com a maior predição para recomendar.

Prática 2 - Sistemas de Recomendação (TensorFlow Recommenders)

1. Importação das Bibliotecas

- **tensorflow_recommenders**: Biblioteca especializada para a construção de sistemas de recomendação no TensorFlow.
- **tensorflow_datasets**: Utilizado para carregar datasets prontos como MovieLens.
- **tensorflow**: Para as funções básicas de TensorFlow e Keras.

2. Obtenção dos Dados

- **tfds.load()**: Carrega dois datasets:
 - **ratings**: Contém classificações dos usuários para filmes.
 - **movies**: Contém os títulos dos filmes disponíveis.
- **map()**: Seleciona as features necessárias, extraíndo o **movie_title** e o **user_id**.

3. Criação dos Vocabulários

- **StringLookup()**: Cria vocabulários para converter **user_id** e **movie_title** em índices inteiros.
 - **adapt()**: Treina os vocabulários nos dados de classificações (**ratings**) e títulos de filmes (**movies**).

4. Definição do Modelo de Recomendação

- **MovieLensModel()**: Define uma classe customizada de recomendação que herda de **tfrs.Model**:
 - **init()**: Inicializa os modelos de usuário e filme, além de configurar a tarefa de recomendação.
 - **compute_loss()**: Calcula a perda comparando os embeddings de usuários e filmes utilizando a tarefa de **retrieval**.

5. Construção do Modelo

- **user_model**: Um modelo Keras que mapeia o `user_id` para um embedding de dimensão 64.
- **movie_model**: Um modelo Keras que mapeia o `movie_title` para um embedding de dimensão 64.
- **task**: Define o objetivo de recomendação com a métrica de top-K fatorada (`factorized_top_k`) para prever recomendações.

6. Treinamento do Modelo

- **model.fit()**: Treina o modelo usando o dataset de classificações por 3 épocas.
- **BruteForce()**: Implementa uma busca exaustiva (brute-force search) para recomendações, onde os embeddings de usuários e filmes são comparados.

7. Recomendações para o Usuário

- **index()**: Gera recomendações para um usuário específico (neste caso, o usuário de ID "42"). O modelo sugere os 3 filmes mais relevantes para esse usuário com base em suas preferências anteriores.